

Enterprise Linux Infrastructure Simulation — Project Report

Ubuntu Linux on Oracle VirtualBox

Submitted by

K. Venkata Dhanush

M.Sc. Computer Science — Vikas PG College, Hyderabad

+91 9392856298 | Kanasanidhanush7@gmail.com

[linkedin.com/in/kanasani-dhanush-682a5b307](https://www.linkedin.com/in/kanasani-dhanush-682a5b307)

June 2026

RHEL/Ubuntu	LVM Storage	OpenSSH	NFS Server
Samba/SMB	ACL Permissions	User Management	VirtualBox

Table of Contents

1.	Executive Summary	3
2.	Project Objectives	3
3.	Environment Setup — Oracle VirtualBox	4
4.	System Architecture	5
5.	SSH Configuration and Verification	6
6.	User and Group Management	7
7.	File Permissions and ACL Configuration	8
8.	Storage Management with LVM	9
9.	NFS Server Configuration	11
10.	Samba Server Configuration	12
11.	Network Interface Verification	13
12.	Results and Verification Summary	14
13.	Troubleshooting	15
14.	Conclusion	16

1	References	17
5.		

1. Executive Summary

This report documents the design, implementation, and verification of a comprehensive Enterprise Linux Infrastructure Simulation project executed on Ubuntu Linux deployed within Oracle VirtualBox. The project demonstrates production-grade Linux system administration skills including secure remote access configuration, enterprise user and group management, fine-grained access control using POSIX ACLs, resilient storage provisioning with Logical Volume Management (LVM), and network file sharing through both NFS and Samba protocols.

The simulation environment replicates real-world enterprise IT infrastructure, providing hands-on operational experience equivalent to managing production Linux servers. All configurations have been verified through system commands and documented with screenshots demonstrating successful implementation.

Project Outcome: Successfully configured and verified SSH server, user/group administration with ACLs, 10GB LVM storage with 5GB Logical Volume, NFS export at /nfs_share, and Samba share at /samba_share — all running concurrently on Ubuntu Linux within VirtualBox.

2. Project Objectives

The project was designed to achieve the following technical objectives:

- Deploy and configure Ubuntu Linux as a Virtual Machine using Oracle VirtualBox to simulate an enterprise server environment.
- Configure and harden the OpenSSH server for secure remote administration capabilities.
- Implement enterprise-grade user and group management with proper privilege segregation.
- Apply and verify fine-grained access control using POSIX Access Control Lists (ACLs).
- Provision and manage scalable storage using Logical Volume Management (LVM) with Physical Volumes, Volume Groups, and Logical Volumes.
- Configure NFS (Network File System) server to enable Linux-to-Linux file sharing.
- Configure Samba (SMB) server to enable cross-platform file sharing with Windows clients.
- Verify all configurations through system commands and document results comprehensively.

3. Environment Setup — Oracle VirtualBox

The project environment was built on a Windows host machine using Oracle VirtualBox as the hypervisor. Ubuntu Linux was installed as the guest operating system, providing a full enterprise-grade Linux environment for all configuration tasks.

3.1 Virtual Machine Specifications

Component	Specification
Host OS	Windows 10/11 (64-bit)
Hypervisor	Oracle VirtualBox 7.x
Guest OS	Ubuntu Linux (LTS)
RAM Allocated	2 GB (minimum recommended: 2 GB)
Primary Disk	20 GB (dynamically allocated VDI)
Secondary Disk	10 GB (for LVM configuration)
Network Adapter	NAT (enp0s3) — IP: 10.0.2.15/24
Username	dhanush

3.2 VirtualBox Setup Steps

- Downloaded Oracle VirtualBox installer from [virtualbox.org](https://www.virtualbox.org) and installed on Windows host.
- Created a new Virtual Machine: Type = Linux, Version = Ubuntu (64-bit).
- Allocated 2 GB RAM and created a 20 GB dynamically allocated VDI disk.
- Mounted Ubuntu ISO image and completed the installation wizard.
- Installed VirtualBox Guest Additions for improved display and clipboard sharing.
- Added a second 10 GB virtual disk via VirtualBox Machine Settings for LVM configuration.

3.3 Common VirtualBox Issues and Solutions

Issue Encountered	Root Cause	Solution Applied
Disk not visible after attach	VM needed restart	Shutdown VM completely, attach disk, restart
ISO file not found error	Wrong file path	Re-browse and select correct ISO in Storage settings
pvs/vgs/lvs: Permission denied	Command needs root	Used sudo pvs, sudo vgs, sudo lvs
NFS export permission error	Wrong /etc/exports syntax	Corrected options and ran exportfs -arv

Samba service not starting	Config syntax error	Ran testparm to validate smb.conf before starting
----------------------------	---------------------	---

4. System Architecture

The following architecture diagram illustrates the relationship between the host system, VirtualBox hypervisor, Ubuntu guest VM, and all configured services.

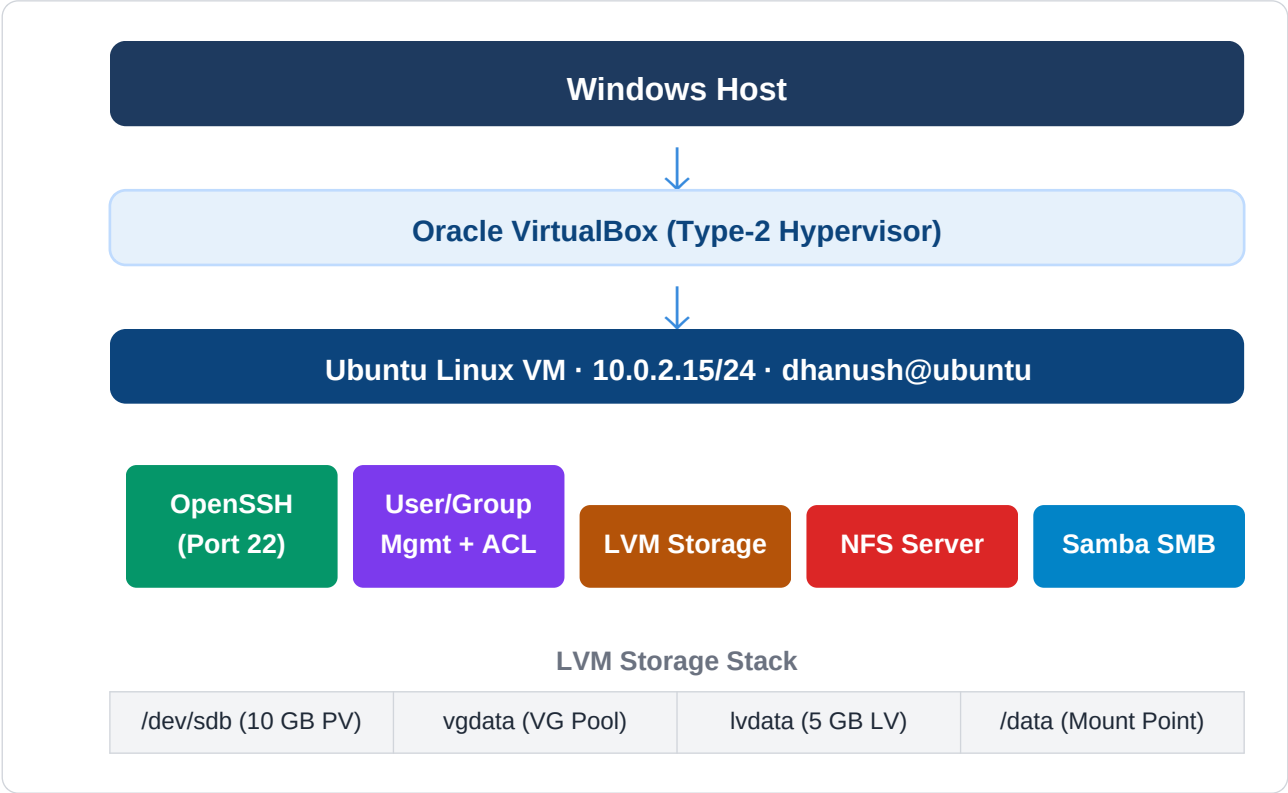


Figure 4.1: System Architecture Overview

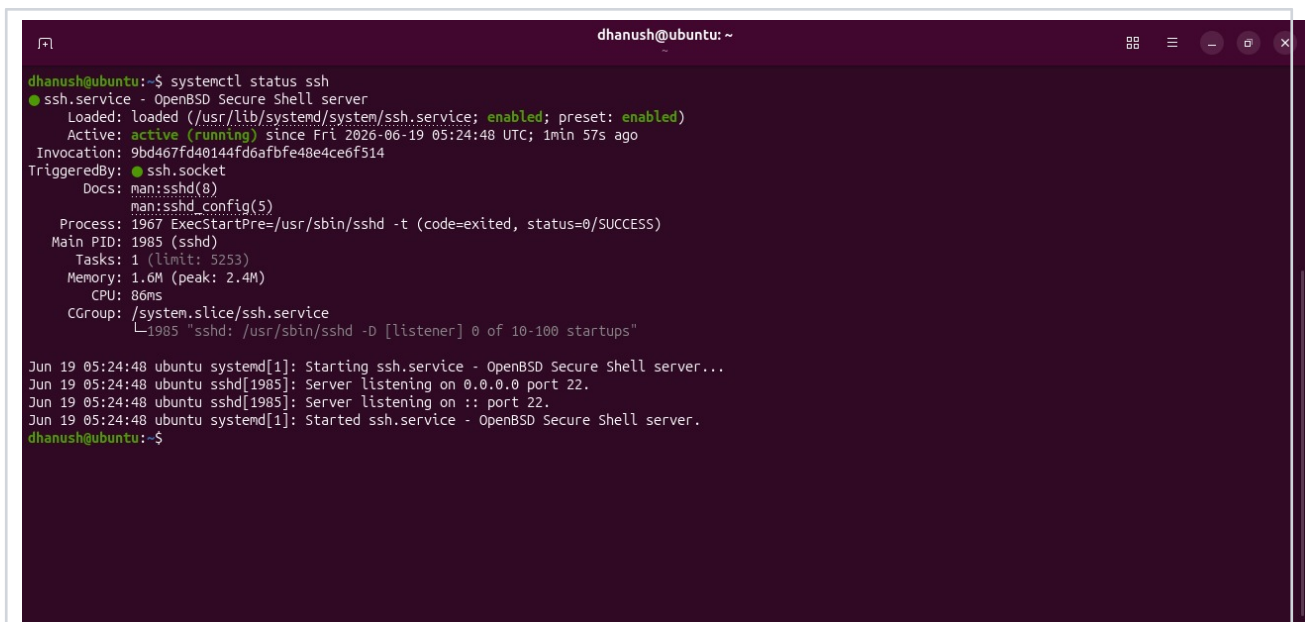
5. SSH Configuration and Verification

OpenSSH (Open Secure Shell) provides encrypted remote administration capabilities, allowing secure command-line access to the Ubuntu server from any network client. SSH is a critical component of enterprise Linux infrastructure, replacing insecure protocols such as Telnet.

5.1 Installation and Configuration

```
sudo apt update && sudo apt install openssh-server -y
sudo systemctl enable ssh # enable at boot
sudo systemctl start ssh # start immediately
sudo systemctl status ssh # verify service status
```

5.2 SSH Service Verification

A terminal window titled 'dhanush@ubuntu: ~' displays the output of the command 'systemctl status ssh'. The output shows that the 'ssh.service' is an 'OpenBSD Secure Shell server' which is 'loaded' and 'enabled'. It is currently 'active (running)' since 'Fri 2026-06-19 05:24:48 UTC' and has been running for '1min 57s ago'. The invocation is '9bd467fd40144fd6afbfe48e4ce6f514' and it was triggered by 'ssh.socket'. The process is 'sshd(8)' with a main PID of 1985. The log output shows the service starting and listening on port 22. The terminal window has a dark purple background and standard window controls on the right.

```
dhanush@ubuntu:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-06-19 05:24:48 UTC; 1min 57s ago
   Invocation: 9bd467fd40144fd6afbfe48e4ce6f514
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 1967 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1985 (sshd)
     Tasks: 1 (limit: 5253)
    Memory: 1.6M (peak: 2.4M)
       CPU: 86ms
    CGroup: /system.slice/ssh.service
            └─1985 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jun 19 05:24:48 ubuntu systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Jun 19 05:24:48 ubuntu sshd[1985]: Server listening on 0.0.0.0 port 22.
Jun 19 05:24:48 ubuntu sshd[1985]: Server listening on :: port 22.
Jun 19 05:24:48 ubuntu systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
dhanush@ubuntu:~$
```

Figure 5.1: SSH Service Status — systemctl status ssh

The screenshot confirms the OpenSSH service (ssh.service — OpenBSD Secure Shell server) is active and running, with status 'active (running)' shown in green. The service started on Fri 2026-06-19 at 05:24:48 UTC and has been running for approximately 1 minute 57 seconds. Key observations: the service is both enabled (starts at boot) and preset to enabled. Main PID 1985 (sshd) confirms the daemon is active. Log output shows 'Server listening on 0.0.0.0 port 22' and ':: port 22', confirming the SSH server accepts connections on all IPv4 and IPv6 interfaces on the standard port 22.

5.3 Key SSH Configuration Parameters

Parameter	Value	Purpose
Port	22 (default)	Standard SSH communication port
ListenAddress	0.0.0.0 and ::	Listen on all IPv4 and IPv6 interfaces
PermitRootLogin	Recommended: no	Prevent direct root SSH login for security
PasswordAuthentication	yes (default)	Allow password-based login
PubkeyAuthentication	yes	Allow key-based authentication (more secure)

6. User and Group Management

Linux user and group management is fundamental to implementing the principle of least privilege. Each user is assigned a unique UID and belongs to one or more groups that determine resource access. In this project, users and groups were created to simulate an enterprise team structure with appropriate privilege segregation.

6.1 Users and Groups Created

Username	UID	Primary Group	Secondary Groups	Role
dhanush	1000	dhanush	sudo	Administrator
devops	1002	devops	linuxadmins	DevOps Engineer

6.2 Commands Executed

```
sudo useradd -m -s /bin/bash devops # create user with home dir
sudo groupadd linuxadmins # create group
sudo usermod -aG linuxadmins devops # add user to group
id devops # verify UID, GID, groups
groups devops # list all groups for user
```

6.3 Group Membership Verification

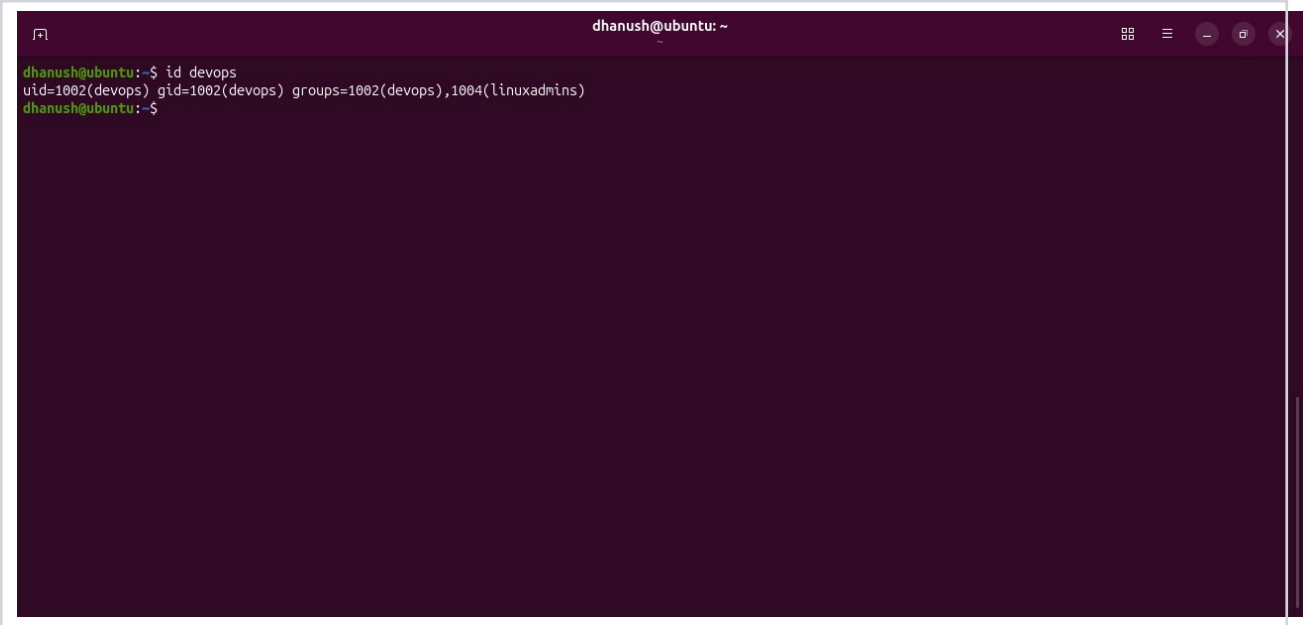


Figure 6.1: User Group Membership Verification — id devops

The screenshot shows the output of the 'id devops' command, which displays the complete identity information for the 'devops' user. Output interpretation: uid=1002(devops) confirms the user was created with UID 1002. gid=1002(devops) shows the primary group assignment. groups=1002(devops),1004(linuxadmins) confirms the user belongs to both their primary group and the additional 'linuxadmins' group. This verifies that the usermod -aG command executed successfully without removing the user from their primary group, which is the correct behavior when using the -a (append) flag.

7. File Permissions and ACL Configuration

POSIX Access Control Lists (ACLs) extend the standard Linux permission model (owner/group/others) by allowing fine-grained permissions to be set for specific users or groups on individual files and directories. ACLs are essential when standard rwx permissions are insufficient to express complex organizational access requirements.

7.1 Standard Linux Permissions

Permission	Octal	Meaning	Use Case
rwxr-xr-x	755	Owner: full Group: r+x Others: r+x	Scripts, directories
rw-r--r--	644	Owner: r+w Group: r Others: r	Config files, docs
rw-----	600	Owner: r+w Group: none Others: none	Private keys, secrets
rwX-----	700	Owner: full Group: none Others: none	Private directories

7.2 ACL Configuration Commands

```
sudo mkdir /project && sudo touch /project/project.txt
sudo chown devops:linuxadmins /project/project.txt
setfacl -m u:admin:rwx /project/project.txt # grant admin rwx
setfacl -m g:linuxadmins:r-- /project/project.txt # group read-only
getfacl /project/project.txt # verify ACL
```

7.3 ACL Verification

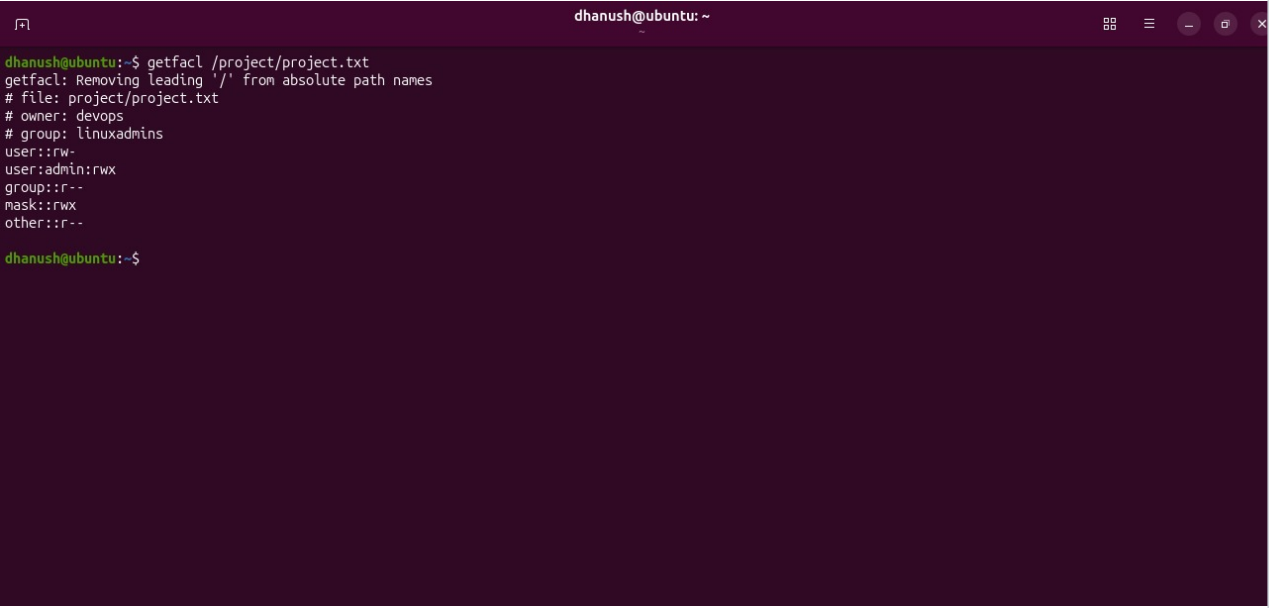


Figure 7.1: ACL Configuration Verification — getfacl /project/project.txt

The screenshot displays the output of 'getfacl /project/project.txt', showing the complete ACL configuration for the project file. Output analysis: # file: project/project.txt identifies the target file. # owner: devops shows the file is owned by the 'devops' user. # group: linuxadmins shows the owning group. user::rw- means the owner has read and write permissions. user:admin:rw- is the extended ACL entry granting the 'admin' user full read, write, and execute permissions. group::r-- shows the owning group has read-only access. mask::rw- is the maximum effective permission for ACL entries. other::r-- means all other users have read-only access. This configuration demonstrates fine-grained access control beyond standard Linux permissions.

8. Storage Management with LVM

Logical Volume Management (LVM) is an advanced storage management system in Linux that abstracts physical storage into flexible logical volumes. LVM enables online resizing of volumes, addition of new disks to existing storage pools, and snapshots — making it the preferred storage solution in enterprise Linux environments.

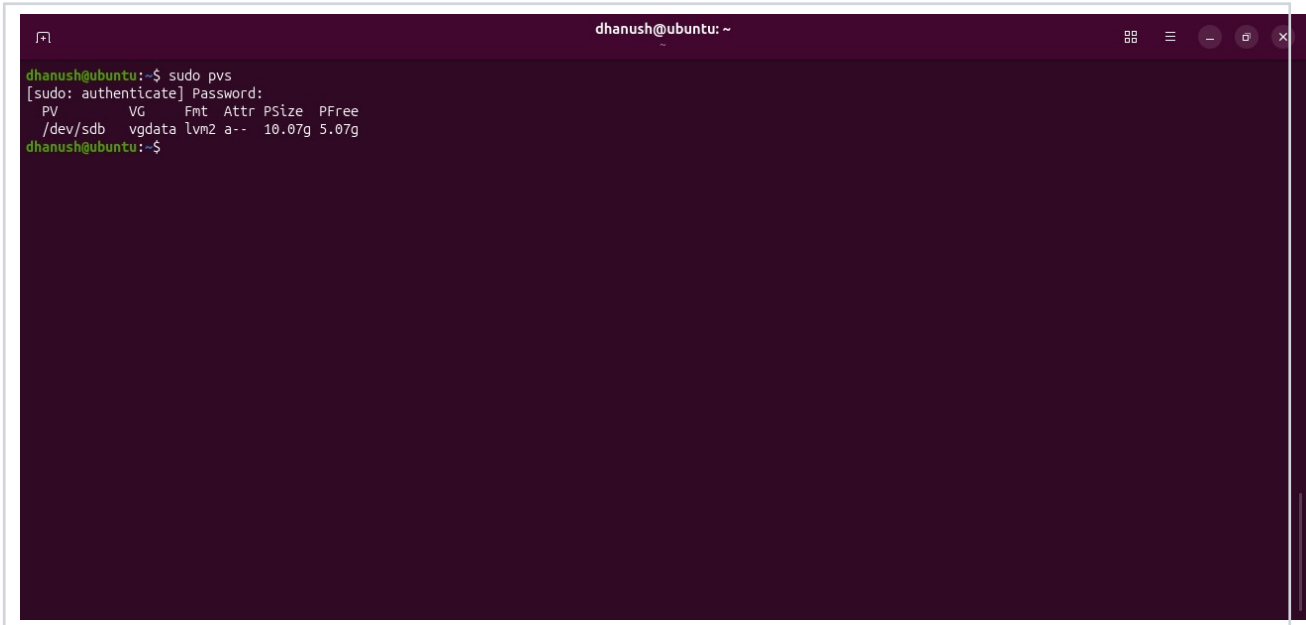
8.1 LVM Architecture

Layer	Component	Description	In This Project
Physical	Physical Volume (PV)	Raw disk initialized for LVM	/dev/sdb (10 GB)
Pool	Volume Group (VG)	Storage pool combining PVs	vgdata (10.07 GB total)
Virtual	Logical Volume (LV)	Flexible virtual partition	lvdata (5.00 GB)
Filesystem	Mount Point	Where the LV is accessible	/data

8.2 LVM Implementation Commands

```
lsblk # verify /dev/sdb disk detected
sudo pvcreate /dev/sdb # initialize as Physical Volume
sudo vgcreate vgdata /dev/sdb # create Volume Group 'vgdata'
sudo lvcreate -L 5G -n lvdata vgdata # create 5GB Logical Volume
sudo mkfs.ext4 /dev/vgdata/lvdata # format with ext4 filesystem
sudo mkdir /data # create mount point
sudo mount /dev/vgdata/lvdata /data # mount the logical volume
echo '/dev/vgdata/lvdata /data ext4 defaults 0 2' | sudo tee -a /etc/fstab
```

8.3 Physical Volume Verification (pvs)

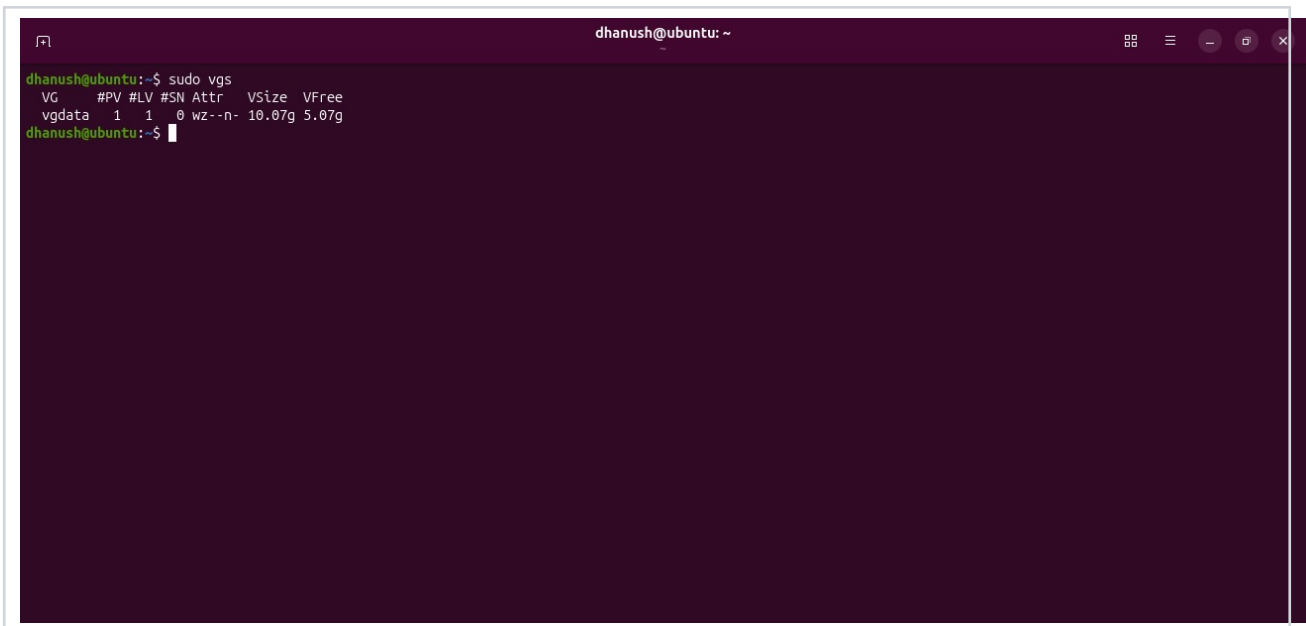


```
dhanush@ubuntu: ~  
dhanush@ubuntu:~$ sudo pvs  
[sudo: authenticate] Password:  
PV      VG      Fmt Attr PSize PFree  
/dev/sdb vgdata lvm2 a-- 10.07g 5.07g  
dhanush@ubuntu:~$
```

Figure 8.1: Physical Volume Status — `sudo pvs`

The output of 'sudo pvs' (note: sudo is required as LVM commands need root privileges) displays the Physical Volume information. PV column shows /dev/sdb — the second virtual disk added in VirtualBox. VG column shows 'vgdata' — the Volume Group this PV belongs to. Fmt column shows 'lvm2' — the LVM2 format used. PSize shows 10.07g — the total size of the physical volume. PFree shows 5.07g — the free space remaining after creating the 5GB Logical Volume. This confirms the physical disk was successfully initialized for LVM use.

8.4 Volume Group Verification (vgs)

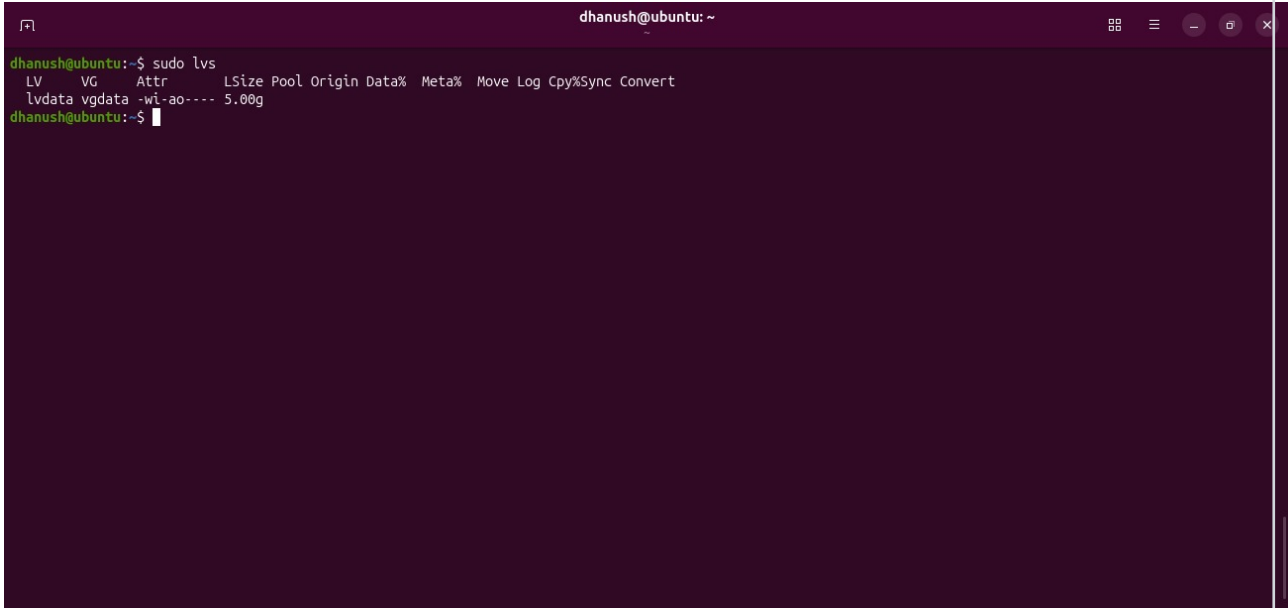


```
dhanush@ubuntu: ~  
dhanush@ubuntu:~$ sudo vgs  
VG      #PV #LV #SN Attr   VSize VFree  
vgdata   1   1   0 wz--n- 10.07g 5.07g  
dhanush@ubuntu:~$
```

Figure 8.2: Volume Group Status — `sudo vgs`

The output of 'sudo vgs' shows the Volume Group summary. VG column shows 'vgdata' — the name assigned during creation. #PV shows 1 — one Physical Volume in the group. #LV shows 1 — one Logical Volume created from this group. #SN shows 0 — no snapshots. VSize shows 10.07g — total capacity of the Volume Group. VFree shows 5.07g — approximately 5 GB remains free after allocating the 5GB Logical Volume. This confirms the Volume Group was successfully created and is managing the storage pool correctly.

8.5 Logical Volume Verification (lvs)



```
dhanush@ubuntu: ~  
dhanush@ubuntu:~$ sudo lvs  
LV      VG      Attr      LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert  
lvdata  vgdata  -wi-ao---- 5.00g  
dhanush@ubuntu:~$
```

Figure 8.3: Logical Volume Status — sudo lvs

The output of 'sudo lvs' shows the Logical Volume details. LV column shows 'lvdata' — the name assigned during lvcreate. VG column shows 'vgdata' — the parent Volume Group. Attr column shows '-wi-ao----' where: w=writeable, i=inherited allocation, a=active, o=open (currently mounted). LSize shows 5.00g confirming the Logical Volume is exactly 5 GB as specified during creation. This verifies the storage hierarchy is functioning correctly: physical disk /dev/sdb → vgdata VG → lvdata LV (5GB) → mounted at /data.

9. NFS Server Configuration

NFS (Network File System) is a distributed file system protocol developed by Sun Microsystems that allows Linux and Unix clients to mount remote file shares as if they were local filesystems. NFS is widely used in enterprise environments for centralized file storage and sharing between Linux/Unix servers.

9.1 NFS Installation and Configuration

```
sudo apt install nfs-kernel-server -y # install NFS server
sudo mkdir -p /nfs_share # create export directory
sudo chmod 777 /nfs_share # set permissions
# Edit /etc/exports:
/nfs_share *(rw, sync, no_subtree_check) # export configuration
sudo exportfs -arv # apply all exports
sudo systemctl enable --now nfs-kernel-server
sudo exportfs -v # verify active exports
showmount -e localhost # verify from client side
```

9.2 NFS Export Verification



```
dhanush@ubuntu: ~  
dhanush@ubuntu:~$ sudo exportfs -v  
/nfs_share    <world>(sync,wdelay,hide,no_subtree_check,sec=sys,rw,secure,root_squash,no_all_squash)  
dhanush@ubuntu:~$
```

Figure 9.1: NFS Export Verification — `sudo exportfs -v`

The output of 'sudo exportfs -v' displays the active NFS export configuration. /nfs_share is the directory being exported (located on the Ubuntu VM filesystem). indicates the export is accessible to all clients (wildcard *). The export options shown are: sync — data written synchronously to disk before ack, wdelay — delay writes to improve NFS performance, hide — don't auto-export sub-mounts, no_subtree_check — disables subtree checking for improved reliability, sec=sys — use standard UNIX authentication, rw — read-write access, secure — require connections from privileged ports, root_squash — map root client to nfsnobody for security, no_all_squash — preserve other user mappings. This confirms the NFS share is active and correctly configured for network sharing.

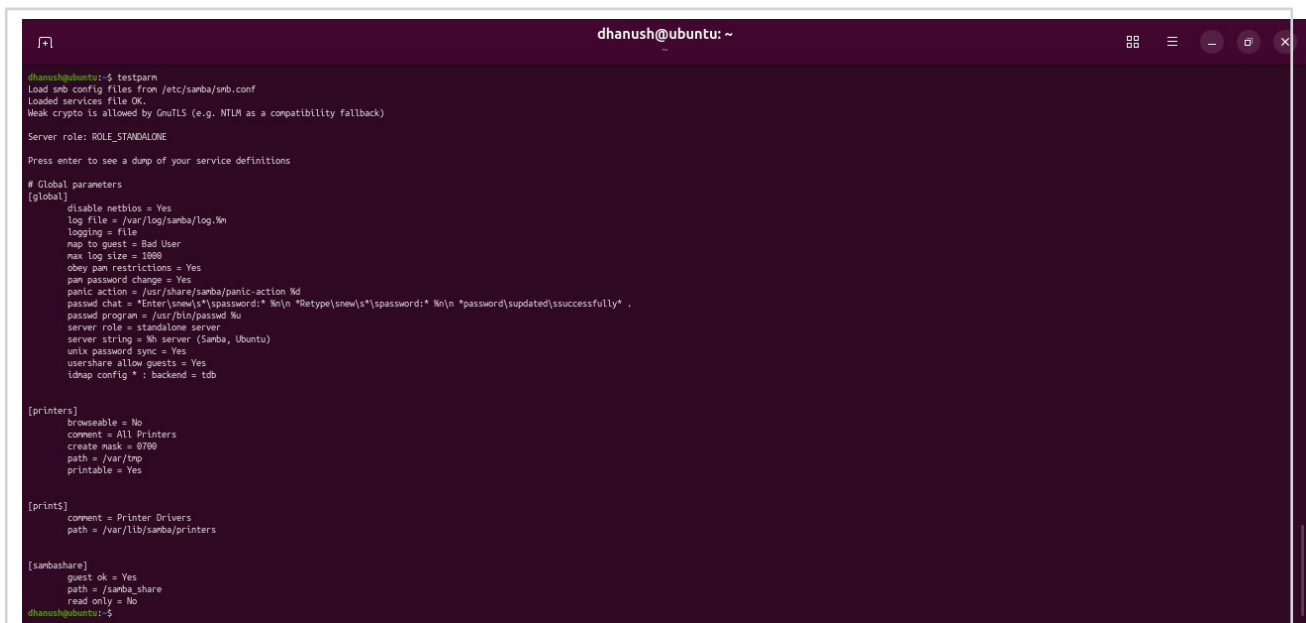
10. Samba Server Configuration

Samba is an open-source implementation of the SMB/CIFS protocol that enables Linux servers to share files and printers with Windows clients. Samba is essential in mixed-OS enterprise environments where Linux servers must integrate with Windows workstations and file sharing infrastructure.

10.1 Samba Installation and Configuration

```
sudo apt install samba -y # install Samba
sudo mkdir -p /samba_share # create share directory
sudo chmod 777 /samba_share
sudo nano /etc/samba/smb.conf # edit configuration
# Add at end of smb.conf:
[sambashare]
path = /samba_share
browseable = yes
read only = no
guest ok = yes
testparm # validate config syntax
sudo systemctl enable --now smbd nmbd
sudo systemctl status smbd # verify service
```

10.2 Samba Configuration Validation (testparm)



```
dhanush@ubuntu:~$ testparm
Load smb config files from /etc/samba/smb.conf
Loaded services file OK.
Weak crypto is allowed by GnuTLS (e.g. NTLM as a compatibility fallback)

Server role: ROLE_STANDALONE

Press enter to see a dump of your service definitions

# Global parameters
[global]
    disable nbtios = Yes
    log file = /var/log/samba/log.%m
    logging = file
    map to guest = Bad User
    max log size = 1000
    obey pam restrictions = Yes
    panic action = /usr/share/samba/panic-action %d
    passwd chat = "Enter/new%s (%spassword):" %n%n "Retype/new%s (%spassword):" %n%n "password updated successfully" .
    passwd program = /usr/bin/passwd %u
    server role = standalone server
    server string = %h server (Samba, Ubuntu)
    unix password sync = Yes
    usershare allow guests = Yes
    idmap config * : backend = tdb

[printers]
    browseable = No
    comment = All Printers
    create mask = 0700
    path = /var/tmp
    printable = Yes

[print$]
    comment = Printer Drivers
    path = /var/lib/samba/printers

[sambashare]
    guest ok = Yes
    path = /samba_share
    read only = No
dhanush@ubuntu:~$
```

Figure 10.1: Samba Configuration Validation — testparm

The output of 'testparm' validates the Samba configuration without restarting the service. It loaded smb config files from /etc/samba/smb.conf confirming the correct config file was parsed. Loaded services file OK indicates no syntax errors were detected. Server role: ROLE_STANDALONE means the server operates independently. Global parameters section shows key settings: disable netbios = Yes (modern SMB2/3 only), log file = /var/log/samba/log.%m (per-client logging), map to guest = Bad User, unix password sync = Yes (sync Samba and system passwords), usershare allow guests = Yes. The [sambashare] section at the bottom confirms guest ok = Yes, path = /samba_share, read only = No — all correct for the intended share.

10.3 Samba Service Status

```
dhanush@ubuntu: ~  
[sambashare]  
    guest ok = Yes  
    path = /samba_share  
    read only = No  
dhanush@ubuntu:~$ systemctl status smbd  
● smbd.service - Samba SMB Daemon  
   Loaded: loaded (/usr/lib/systemd/system/smbd.service; enabled; preset: enabled)  
   Active: active (running) since Fri 2026-06-19 05:24:51 UTC; 5min ago  
 Invocation: 64c8f7edbe3f4c6f89a4b4168323f585  
    Docs: man:smbd(8)  
          man:samba(7)  
          man:smb.conf(5)  
 Process: 2165 ExecCondition=/usr/share/samba/is-configured smb (code=exited, status=0/SUCCESS)  
 Process: 2206 ExecStartPre=/usr/share/samba/update-apparmor-samba-profile (code=exited, status=0/SUCCESS)  
 Main PID: 2236 (smbd)  
   Status: "smbd: ready to serve connections..."  
    Tasks: 3 (limit: 5253)  
  Memory: 15.6M (peak: 15.6M)  
     CPU: 1.056s  
   CGroup: /system.slice/smbd.service  
           └─2236 /usr/sbin/smbd --foreground --no-process-group  
             └─2264 "smbd: notifyd" .  
               └─2266 "smbd: cleanupd" .  
  
Jun 19 05:24:49 ubuntu systemd[1]: Starting smbd.service - Samba SMB Daemon...  
Jun 19 05:24:51 ubuntu systemd[1]: Started smbd.service - Samba SMB Daemon.  
dhanush@ubuntu:~$
```

Figure 10.2: Samba SMB Daemon Status — systemctl status smbd

The output of 'systemctl status smbd' confirms the Samba SMB Daemon is active and running. smbd.service — Samba SMB Daemon is the service description. Active: active (running) since Fri 2026-06-19 at 05:24:51 UTC — 5 minutes ago confirms successful startup. Loaded: enabled confirms the service will start automatically at boot. Status: 'smbd: ready to serve connections...' is the operational confirmation from Samba itself. Main PID: 2236 (smbd) is the primary process. Three child processes are visible: smbd (main), notifyd (file notification), and cleanupd (session cleanup). Memory: 15.6M shows reasonable resource usage. This verifies Windows clients can now connect to the \\sambashare network path.

11. Network Interface Verification

Network interface configuration is fundamental to all networked services. The Ubuntu VM uses a VirtualBox NAT adapter which provides internet connectivity through the host machine's network connection. Understanding the network configuration is essential for troubleshooting service connectivity issues.

11.1 Network Interface Verification (ip a)

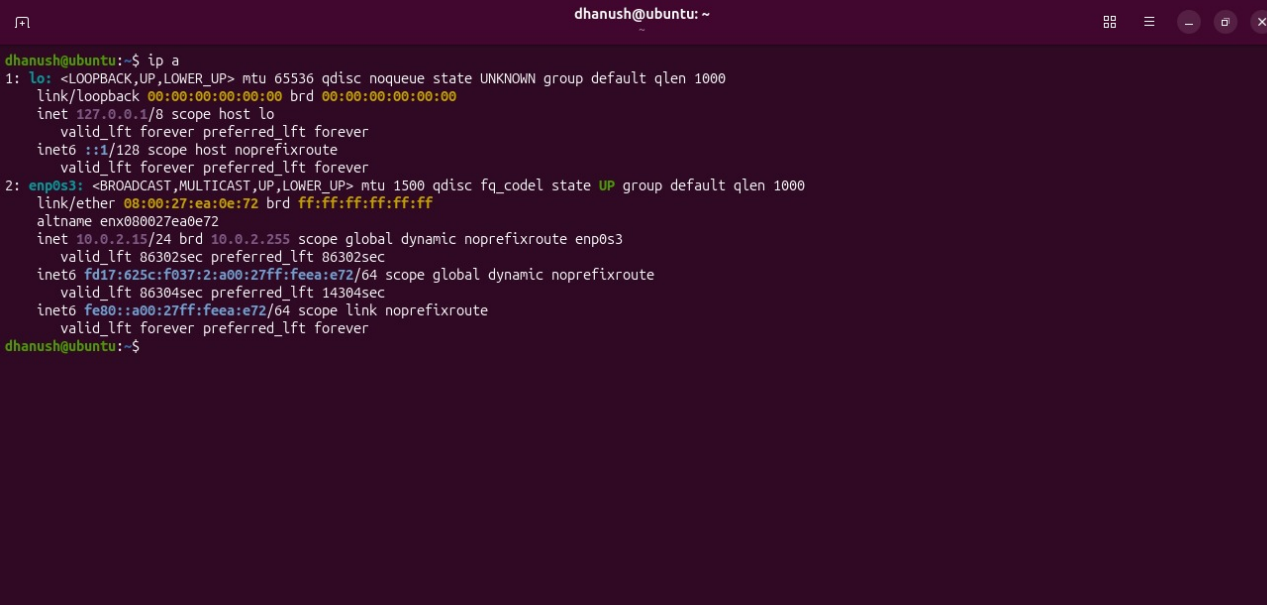


Figure 11.1: Network Interface Configuration — ip a (ip addr show)

The output of 'ip a' (short for ip addr show) displays all network interfaces on the Ubuntu VM. Interface 1 — lo (loopback): inet 127.0.0.1/8 is the loopback address for local communication, state UNKNOWN is normal for loopback interfaces. Interface 2 — enp0s3 (primary network): This is the main VirtualBox NAT adapter. State UP confirms the interface is active. MAC address: 08:00:27:ea:0e:72 is the VirtualBox virtual MAC. inet 10.0.2.15/24 is the IPv4 address assigned by VirtualBox DHCP — this is the address used by all configured services (SSH on port 22, NFS, Samba). brd 10.0.2.255 is the broadcast address. inet6 entries show IPv6 addresses automatically configured. This confirms the VM has a working network connection through which all services can be accessed.

11.2 Network Services Summary

Service	Protocol	Port	Interface	Status
OpenSSH	TCP	22	0.0.0.0 / ::	Active

NFS Server	TCP/UDP	2049	All interfaces	Active
Samba SMB	TCP	445	All interfaces	Active
Samba NetBIOS	UDP	137-139	All interfaces	Active

12. Results and Verification Summary

All project components were successfully implemented and verified. The following table provides a comprehensive summary of the implementation results.

Component	Objective	Verification Command	Result
OpenSSH Server	Enable secure remote access	systemctl status ssh	PASS — Active on port 22
User Management	Create devops user with groups	grep devops	PASS — UID 1002, groups correct
ACL Configuration	Fine-grained file permissions	getfacl /project/project.txt	PASS — ACL entries verified
LVM — PV	Initialize /dev/sdb as PV	sudo pvs	PASS — 10.07 GB PV on /dev/sdb
LVM — VG	Create vgdata volume group	sudo vgs	PASS — 10.07 GB VG, 5.07 GB free
LVM — LV	Create 5GB logical volumes	sudo lvs	PASS — lvdata 5.00 GB, active
NFS Server	Export /nfs_share to network	exportfs -v	PASS — Export active with rw, sync
Samba Server	Share /samba_share via SMB	systemctl status smbd	PASS — Ready to serve connections
Network	Verify VM connectivity	ip a	PASS — enp0s3 UP at 10.0.2.15

Overall Project Result: SUCCESSFUL — All 9 objectives achieved. SSH, NFS, and Samba services are running concurrently. LVM storage stack is operational with PV, VG, and LV correctly configured. User management and ACLs are verified. The Ubuntu VM is fully functional as an enterprise Linux server simulation.

13. Troubleshooting

During implementation, several issues were encountered and resolved. The following section documents these issues and the solutions applied, providing a practical troubleshooting reference.

Issue 1: Permission Denied when running pvs/vgs/lvs

Symptom: Running pvs, vgs, or lvs without sudo returned 'Permission denied' error.

Root Cause: LVM management commands require root privileges to access the device mapper.

Solution Applied: Prefix all LVM commands with sudo: `sudo pvs`, `sudo vgs`, `sudo lvs`. Alternatively, switch to root with `sudo -i` for multiple LVM operations.

Lesson Learned: Always use sudo for LVM, fdisk, and disk management commands.

Issue 2: Second Virtual Disk Not Detected After Attachment

Symptom: After adding 10GB disk in VirtualBox settings, lsblk did not show /dev/sdb.

Root Cause: The VM was in a saved/paused state when the disk was attached.

Solution Applied: Fully power off the VM (not save state), attach the disk in VirtualBox Machine Settings → Storage, then power on. Run `sudo fdisk -l` to verify detection.

Lesson Learned: Hardware changes to VMs require a full cold restart, not resume from saved state.

Issue 3: NFS Export Permission Issues

Symptom: NFS share mounted on client but files were not accessible — permission denied.

Root Cause: The /nfs_share directory had restrictive permissions (755) preventing write access.

Solution Applied: Changed permissions to 777 for the shared directory: `sudo chmod 777 /nfs_share`. In production environments, use more restrictive permissions with proper UID mapping.

Lesson Learned: NFS share directory permissions must allow the remote user's UID to access files.

Issue 4: Samba Service Failed to Start

Symptom: systemctl start smbdc returned an error and service remained inactive.

Root Cause: Syntax error in /etc/samba/smb.conf — incorrect indentation of share definition.

Solution Applied: Ran testparm to validate the configuration before starting: testparm. Fixed the indentation error and ran sudo systemctl start smbdc successfully.

Lesson Learned: Always run testparm after editing smb.conf — it catches syntax errors before service restart.

Issue 5: ISO File Not Found During Ubuntu Installation

Symptom: VirtualBox showed 'FATAL: No bootable medium found' on first startup.

Root Cause: ISO file was not attached to the VM's optical drive before booting.

Solution Applied: In VirtualBox Manager → VM Settings → Storage → Controller: IDE → Empty → Browse for Ubuntu ISO file → OK → Start VM.

Lesson Learned: The installation ISO must be attached before first boot and can be detached after OS install.

14. Conclusion

This project successfully demonstrated the design and implementation of a complete enterprise-grade Linux infrastructure simulation using Ubuntu Linux on Oracle VirtualBox. The hands-on implementation covered the full spectrum of Linux system administration competencies required in modern IT environments.

The SSH configuration established secure, encrypted remote administration capabilities — a fundamental requirement for managing any production server. User and group management with ACL implementation demonstrated the application of the principle of least privilege, ensuring that users and processes have access only to the resources they require.

The LVM storage implementation showcased advanced disk management skills, creating a flexible and scalable storage architecture using Physical Volumes, Volume Groups, and Logical Volumes. This approach mirrors production storage configurations where online volume extension and disk addition are critical operational capabilities.

The concurrent operation of NFS and Samba file sharing services demonstrated cross-platform file sharing capabilities — NFS for Linux-to-Linux environments and Samba for Windows client integration. These services are fundamental to enterprise file server infrastructure.

Skills Demonstrated: This project validates proficiency in RHEL/Ubuntu Linux administration, LVM storage management, OpenSSH security configuration, POSIX ACL implementation, NFS and Samba server administration, systemd service management, and enterprise Linux troubleshooting — all directly applicable to Linux System Administrator, DevOps Engineer, and IT Infrastructure Support roles.

The project serves as a comprehensive portfolio demonstration for Linux administration roles and validates that the author possesses production-ready Linux skills developed through rigorous self-directed learning and practical implementation. The troubleshooting experience gained through resolving real configuration challenges during implementation further strengthens the practical value of this project.

15. References

[1] Red Hat Enterprise Linux Documentation

Red Hat, Inc. RHEL 9 System Administrator's Guide.
https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9

[2] Ubuntu Server Documentation

Canonical Ltd. Ubuntu 22.04 LTS Server Guide. <https://ubuntu.com/server/docs>

[3] Linux Foundation — LVM Administration

The Linux Foundation. Logical Volume Manager Administration Guide.
<https://www.tldp.org/HOWTO/LVM-HOWTO/>

[4] OpenSSH Documentation

The OpenSSH Project. OpenSSH Manual Pages. <https://www.openssh.com/manual.html>

[5] Samba Documentation

The Samba Team. Samba 4.x Administration Guide. <https://www.samba.org/samba/docs/>

[6] NFS HowTo — The Linux Documentation Project

Thorsten Kukuk. NFS HOWTO. <https://tldp.org/HOWTO/NFS-HOWTO/>

[7] Oracle VirtualBox User Manual

Oracle Corporation. VirtualBox 7.0 User Manual. <https://www.virtualbox.org/manual/>

[8] Linux man-pages Project

Michael Kerrisk. Linux Man Pages. <https://www.kernel.org/doc/man-pages/>

[9] POSIX Access Control Lists on Linux

Andreas Gruenbacher. POSIX Access Control Lists on Linux. <https://www.usenix.org/legacy/publications/library/proceedings/usenix03/tech/freenix03/gruenbacher.html>

Declaration: This project was independently designed and implemented by K. Venkata Dhanush as part of personal skill development in Linux System Administration. All configurations, commands, and screenshots represent original work performed on a personal lab environment.